

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	G.Dhanusha Sree	Reg. No.:	192325012
Section / Batch:	2023	Date:	14/07/2026

Code of Conduct

I G.Dhanusha Sree (192325012) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G.Dhanusha Sree

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Problem Understanding

A software company is developing a user registration system that validates user credentials before creating an account. The system checks whether the email address, password, and mobile number follow the required format using Regular Expressions. This ensures secure registration and prevents invalid user data.

Algorithm

1. Import the re module.
2. Accept Email, Password, and Mobile Number from the user.
3. Validate Email using a Regular Expression.
4. Validate Password based on:
 1. Minimum 8 characters
 2. At least one uppercase letter
 3. At least one lowercase letter
 4. At least one digit
 5. At least one special character
5. Validate Mobile Number.
6. Display validation results.

PROGRAM:

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"

password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$"

mobile_pattern = r"^[6-9]\d{9}$"

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
```

```
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

PROGRAM:

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"

password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$"

mobile_pattern = r"^[6-9]\d{9}$"

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPEED CHECKER

PS C:\Users\Asus\Documents\NLP Program> & C:\Users\Asus\AppData\Local\Programs\Python\Python39\python.exe C:\Users\Asus\Documents\NLP Program\programs.py
Enter Email: dhanushasree@gmail.com
Enter Password: 1234
Enter Mobile Number: 98078890
Valid Email
Weak Password
Invalid Mobile Number
PS C:\Users\Asus\Documents\NLP Program>
```

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

Problem Understanding

A Deterministic Finite Automata (DFA) is used to recognize whether an input string belongs to a particular language. This program simulates a DFA that accepts strings ending with "ab" by processing each character and moving between predefined states.

Algorithm

1. Define DFA states and transitions.
2. Read the input string.
3. Start from the initial state.
4. Process each character according to the transition table.
5. Display the transition path.
6. If the final state is reached, accept the string; otherwise reject it.

PROGRAM:

```
transitions = {
    ('q0', 'a'): 'q1',
    ('q0', 'b'): 'q0',
    ('q1', 'a'): 'q1',
    ('q1', 'b'): 'q2',
    ('q2', 'a'): 'q1',
    ('q2', 'b'): 'q0'
}

state = 'q0'
final_state = 'q2'

string = input("Enter String: ")

path = [state]

for ch in string:
    state = transitions[(state, ch)]
    path.append(state)

print("Transition Path:")
print(" -> ".join(path))

if state == final_state:
```

```
    print("Accepted")
else:
    print("Rejected")
```

```
✓ transitions = {
    ('q0', 'a'): 'q1',
    ('q0', 'b'): 'q0',
    ('q1', 'a'): 'q1',
    ('q1', 'b'): 'q2',
    ('q2', 'a'): 'q1',
    ('q2', 'b'): 'q0'
}

state = 'q0'
final_state = 'q2'

string = input("Enter String: ")

path = [state]

✓ for ch in string:
    state = transitions[(state, ch)]
    path.append(state)

print("Transition Path:")
print(" -> ".join(path))

✓ if state == final_state:
    print("Accepted")
✓ else:
    print("Rejected")
```

OUTPUT:

```
PS C:\Users\Asus\Documents\NLP Program> & C:\Users\Asus\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\Asus\Documents\NLP Program\programs.py
q0 -> q1 -> q1 -> q1 -> q2
Accepted
PS C:\Users\Asus\Documents\NLP Program> 
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

Problem Understanding

A text search engine is required to identify different types of patterns such as dates, phone numbers, hashtags, mentions, prefixes, and suffixes using Regular Expressions. This improves searching and information extraction from text.

Algorithm

1. Store the input text.
2. Display the search menu.
3. Accept the user's choice.
4. Apply the appropriate Regular Expression.
5. Display all matching patterns.

Program:

```
import re
text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

print("1.Date")
print("2.Phone Number")
print("3.Hashtag")
print("4.Mention")
print("5.Prefix")
print("6.Suffix")
choice = int(input("Enter Choice: "))

if choice == 1:
    print(re.findall(r"\d{2}^\d{2}^\d{4}", text))
elif choice == 2:
    print(re.findall(r"[6-9]\d{9}", text))
elif choice == 3:
    print(re.findall(r"#\w+", text))
elif choice == 4:
    print(re.findall(r"@ \w+", text))
elif choice == 5:
    word = input("Enter Prefix: ")
    print(re.findall(r"\b" + word + r"\w*", text, re.I))
elif choice == 6:
```

```
word = input("Enter Suffix: ")
print(re.findall(r"\w*" + word + r"\b", text, re.I))
```

```
1  import re
2  text = """
3  Meeting on 12/09/2026
4  Call 9876543210
5  #NLP
6  @OpenAI
7  natural language processing
8  """
9  print("1.Date")
10 print("2.Phone Number")
11 print("3.Hashtag")
12 print("4.Mention")
13 print("5.Prefix")
14 print("6.Suffix")
15 choice = int(input("Enter Choice: "))
16 if choice == 1:
17     print(re.findall(r"\d{2}/\d{2}/\d{4}", text))
18 elif choice == 2:
19     print(re.findall(r"[6-9]\d{9}", text))
20
21 elif choice == 3:
22     print(re.findall(r"#\w+", text))
23
24 elif choice == 4:
25     print(re.findall(r"@w+", text))
26 elif choice == 5:
27     word = input("Enter Prefix: ")
28     print(re.findall(r"\b" + word + r"\w*", text, re.I))
29 elif choice == 6:
30     word = input("Enter Suffix: ")
31     print(re.findall(r"\w*" + word + r"\b", text, re.I))
32
33
34
```

OUTPUT:

```
PS C:\Users\Asus\Documents\NLP Program> & C:\Users\Asus\AppData\Local\Programs\Python\Python311\python.exe C:\Users\Asus\Documents\NLP Program\programs.py
1.Date
2.Phone Number
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 1
['12/09/2026']
PS C:\Users\Asus\Documents\NLP Program> & C:\Users\Asus\AppData\Local\Programs\Python\Python311\python.exe C:\Users\Asus\Documents\NLP Program\programs.py
1.Date
2.Phone Number
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 2
['9876543210']
```